

Redis 持久化

Redis的持久化机制

Redis 提供两种持久化机制： RDB（默认） 和 AOF 机制，在 Redis4.0 之后还提供了混合持久化

1. RDB: 是指内存快照，指内存中的数据在某一个时刻的状态记录。
2. AOF: AOF日志 里记录的是 Redis 收到的每一条命令，这些命令是以文本形式保存的。
3. 混合持久化: RDB 以一定的频率执行，在两次快照之间，使用 AOF 日志记录这期间的所有命令操作

RDB 持久化

1. 内存快照，就是指内存中的数据在某一个时刻的状态记录，**保存到硬盘中，对应产生的数据文件为dump.rdb**，和 AOF 相比，RDB 记录的是某一时刻的数据，并不是操作，所以，在做数据恢复时，我们可以直接把 RDB 文件读入内存，不是重新执行命令。
2. Redis 的数据都在内存中，为了提供所有数据的可靠性保证，它执行的是**全量快照**，也就是说，把内存中的所有数据都记录到磁盘中。

RDB 的优缺点

优点

1. 整个Redis数据库将只包含一个文件dump.rdb，方便持久化。
2. 容灾性好，方便备份。
3. 性能最大化，fork 子进程来完成写操作。使用单独子进程来进行持久化，主进程不会进行任何 IO 操作,保证了Redis的高性能

4. 相对于数据集大时, 比AOF的启动效率更高。

缺点

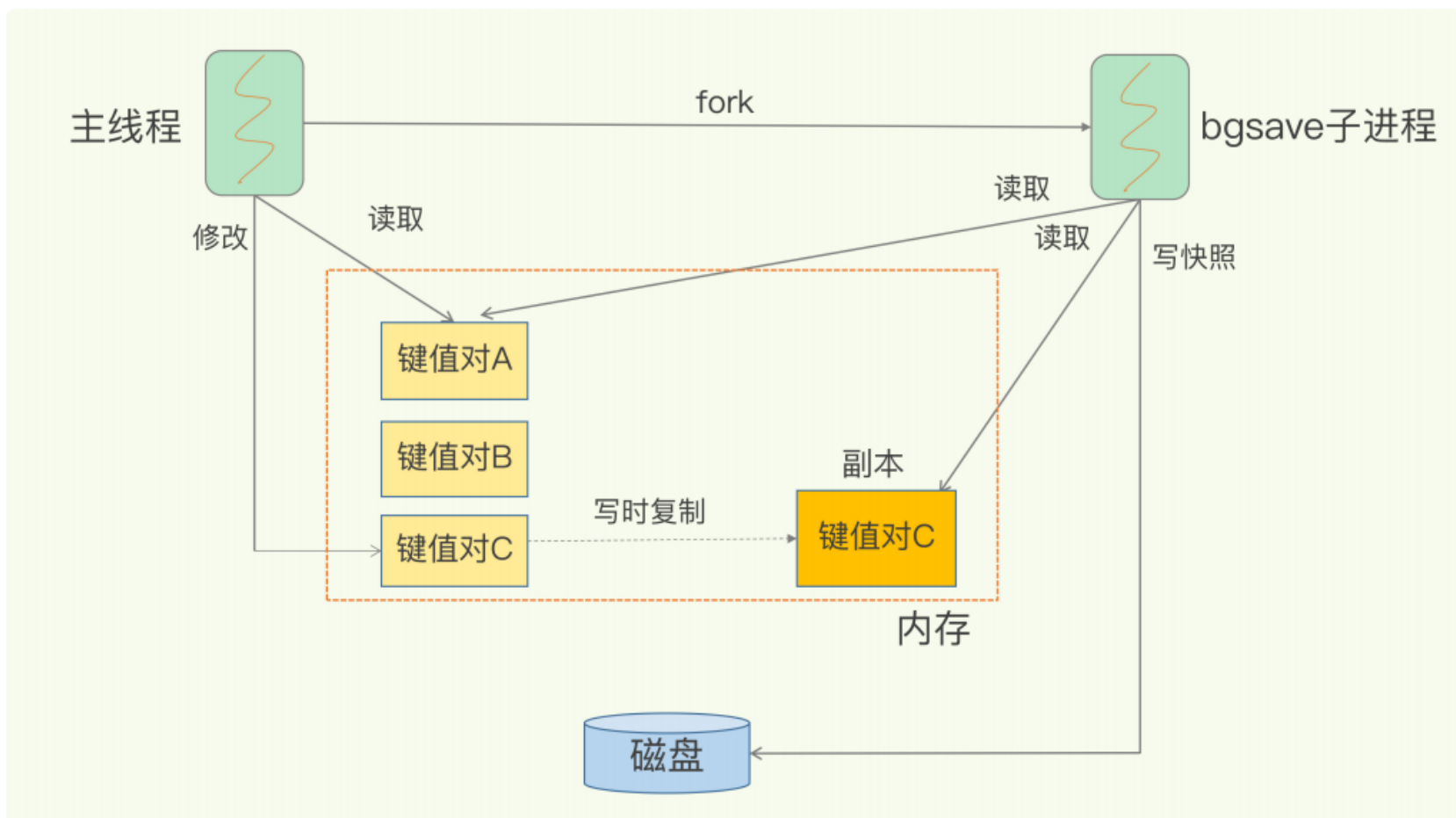
1. 数据安全性低。RDB是间隔一段时间进行持久化, 如果持久化之间redis发生故障, 会发生数据丢失。所以这种方式更适合数据要求不严谨的时候
2. 由于RDB是通过fork子进程来协助完成数据持久化工作的, 因此,如果当数据集较大时, 可能会导致整个服务器停止服务几百毫秒, 甚至是1秒钟, 会占用cpu。

RDB 快照时运行修改数据吗

1. 快照为了保证快照完整性不能进行写操作, 但是为了快照而暂停写操作, 肯定是不能接受的。
2. Redis 就会借助操作系统提供的写时复制技术 (Copy-On-Write, COW) , 在执行快照的同时, 正常处理写操作。

RDB 快照时修改数据过程

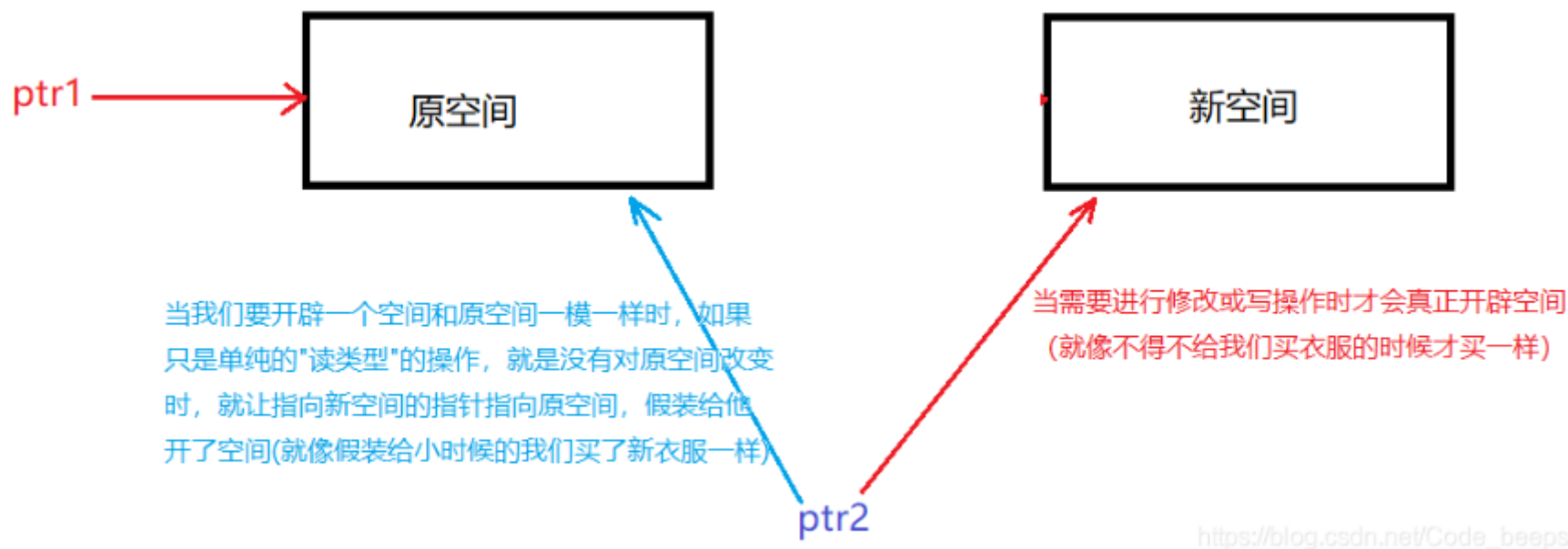
1. bgsave 子进程是由主线程 fork 生成的, 共享主线程的所有内存数据。bgsave 子进程运行后, 开始读取主线程的内存数据, 并把它们写入 RDB 文件。
2. 如果主线程对这些数据是读操作, 主线程和bgsave 子进程相互不影响。
3. 如果主线程要进行写操作修稿某个数据, 这块数据就会被复制一份, 生成该数据的副本bgsave 子进程会把这个副本数据写入 RDB 文件, 而在这个过程中, 主线程仍然可以直接修改原来的数据



写时复制技术

1. 写入时复制(Copy-on-write)是存储系统中使用的基本更新策略之一。
2. COW核心思想就是如果有多个调用者同时请求相同资源（如内存或磁盘上的数据存储服务），他们会共同获取相同的指针指向相同的资源，直到某个调用者试图修改资源的内容时，系统才会真正复制一份专用副本给该调用者，而其他调用者所见到的最初的资源仍然保持不变。

3. COW 是一种优化策略，fork 子进程是linux提供的创建新线程的方法，大多数的 fork 实现借用了COW 策略来节省内存空间。



RDB 的执行频率

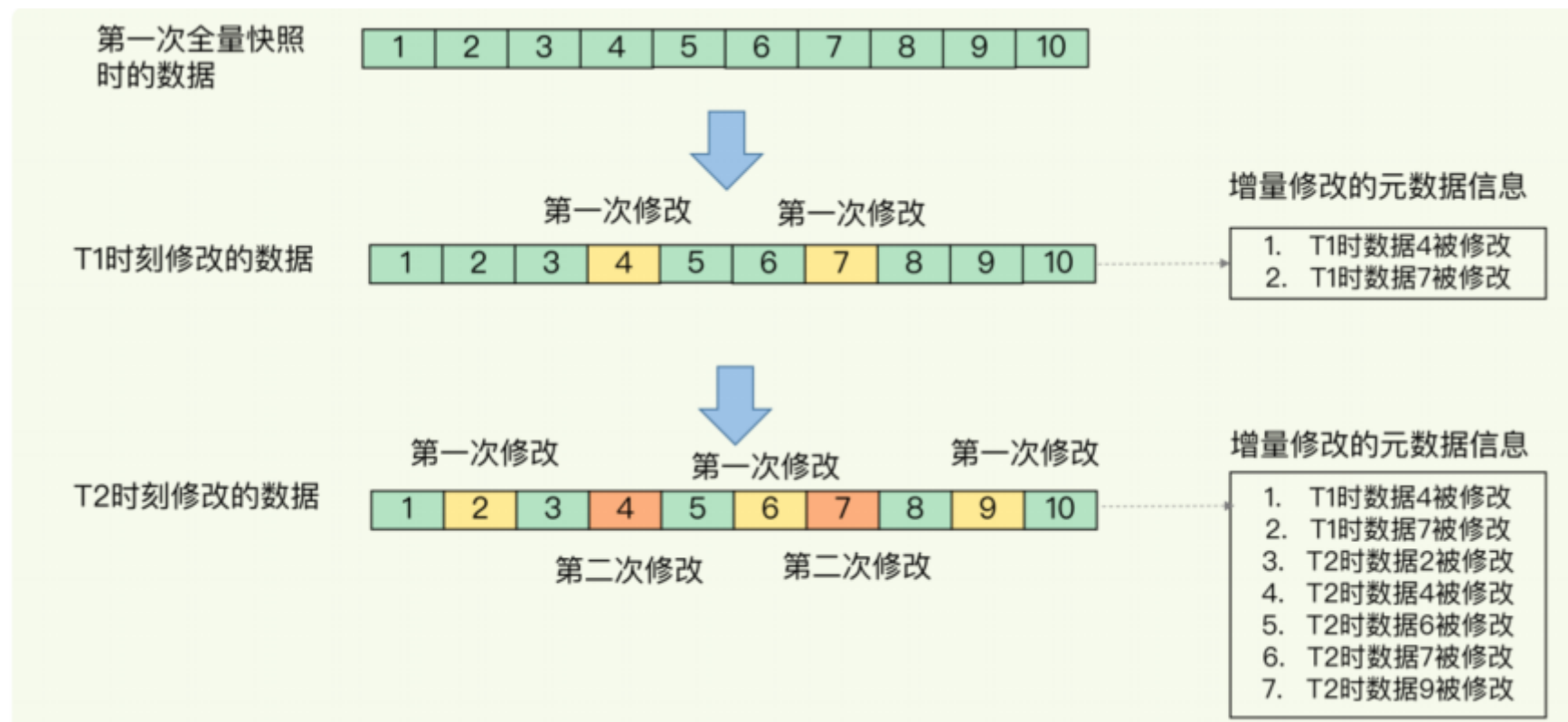
1. 频率过低，某一时间发生了宕机了，那么就会丢失较多数据
2. 频率过高，虽然bgsave执行时不阻塞主线程，但是如果频繁地执行全量快照，也会带来两方面的开销
 1. 频繁将全量数据写入磁盘，会给磁盘带来很大压力，多个快照竞争有限的磁盘带宽，前一个快照还没有做完，后一个又开始做了，容易造成恶性循环。
 2. bgsave 子进程需要通过 fork 操作从主线程创建出来。虽然，子进程在创建后不会再阻塞主线程，但是，fork 这个创建过程本身会阻塞主线程，而且主线程的内存越大，阻塞时间越长。如果频繁 fork 出 bgsave 子进程，这就会频繁阻塞主线程了。

增量快照

由于快照的频率不好把握，所以可以通过增量快照的方式解决全量快照的弊端。

增量快照：做了一次全量快照后，后续的快照只对修改的数据进行快照记录，这样可以避免每次全量快照的开销。

增量快照会带来额外的空间开销，因为需要额外的元数据信息去记录哪些数据被修改，这对于内存资源宝贵的 Redis 来说，有些得不偿失。



AOF 持久化

1. AOF持久化(即Append Only File持久化)，以日志的形式记录服务器所处理的每一个写，删除操作，而查询操作不会记录。

2. AOF采用的是写后日志的方式，Redis先执行命令把数据写入内存，然后再记录日志到文件中。

3. 在redis的默认配置中AOF持久化机制是没有开启的，需要在配置文件中开启

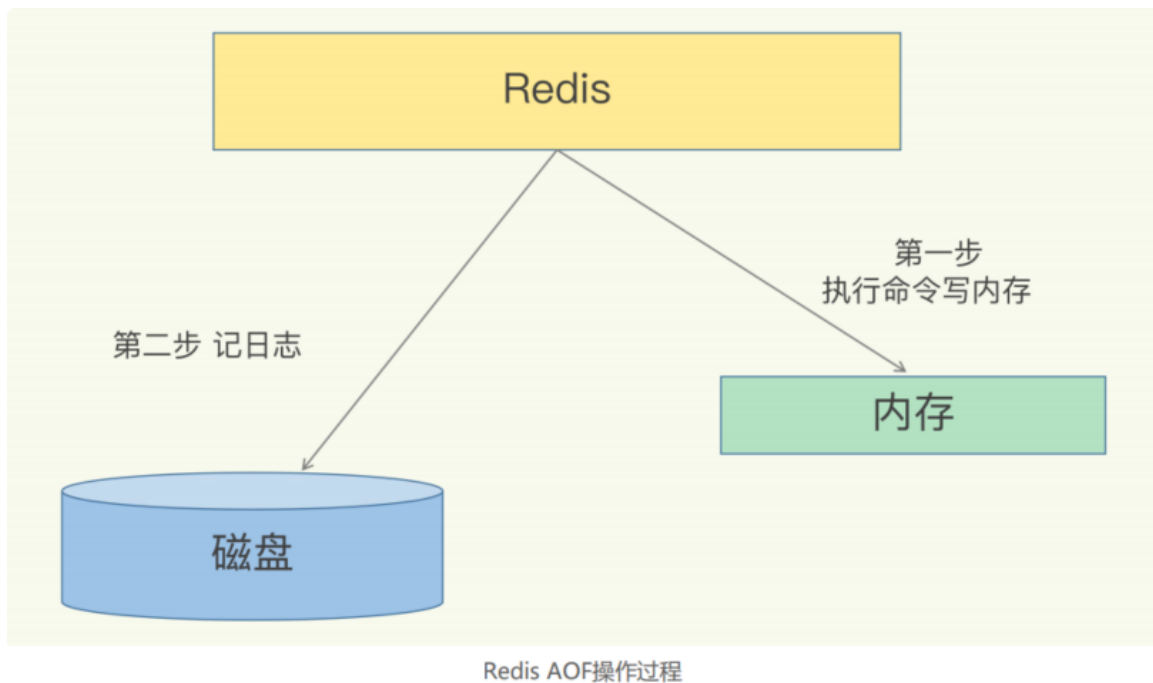
1. 修改 `appendonly yes` 开启持久化

2. 修改 `appendfilename "appendonly.aof"` 指定生成文件名称

AOF 为什么要采用后写日志呢？

AOF采用的是 "写后日志"的方式，我们平时用的MySQL则采用的是 "写前日志"。

1. **可以避免出现记录错误命令的情况**。Redis在向AOF里面记录日志的时候，并不会先去对这些命令进行语法检查。所以，如果先记日志再执行命令的话，若命令有错，并在记日志之后发生宕机，日志中就有可能记录了错误的命令，Redis在使用日志恢复数据时，就可能出错。
2. **它在命令执行后才记录日志**，不会阻塞当前的写操作。



后写日志的弊端

后写日志主要有两个风险可能会发生：

1. 数据可能会丢失：如果 Redis 刚执行完命令，此时发生故障宕机，会导致这条命令存在丢失的风险。
2. 可能阻塞其他操作：AOF 日志其实也是在主线程中执行，所以当 Redis 把日志文件写入磁盘的时候，还是会阻塞后续的操作无法执行。

AOF 的优缺点

优点

1. 数据安全
2. 通过append模式写文件，即使中途服务器宕机也不会破坏已经存在的内容，可以通过redis-check-aof工具解决数据一致性问题。
3. AOF 机制的rewrite模式。定期对AOF文件进行重写，以达到压缩的目的

缺点

1. AOF文件比RDB文件大，且恢复速度慢。
2. 数据集大的时候，比RDB启动效率低。
3. 运行效率没有RDB高

AOF 日志写回磁盘的策略

AOF 机制给我们提供了三个选择，也就是 AOF 配置项 `appendfsync` 的三个可选值

1. Always，同步写回：每个写命令执行完，立马同步地将日志写回磁盘；
2. Everysec，每秒写回：每个写命令执行完，只是先把日志写到 AOF 文件的内存缓冲区，每隔一秒把缓冲区中的内容写入磁盘；
3. No，操作系统控制的写回：每个写命令执行完，只是先把日志写到 AOF 文件的内存缓冲区，由操作系统决定何时将缓冲区内容写回磁盘；

配置项	写回时机	优点	缺点
Always	同步写回	可靠性高，数据基本不丢失	每个写命令都要落盘，性能影响较大
Everysec	每秒写回	性能适中	宕机时丢失1秒内的数据
No	操作系统控制的写回	性能好	宕机时丢失数据较多

AOF 如何选择写回策略

1. 想要获得高性能，就选择 No 策略；
2. 如果想要得到高可靠性保证，就选择Always 策略；
3. 如果允许数据有一点丢失，又希望性能别受太大影响的话，那么就选择 Everysec 策略；

AOF日志过大的危害

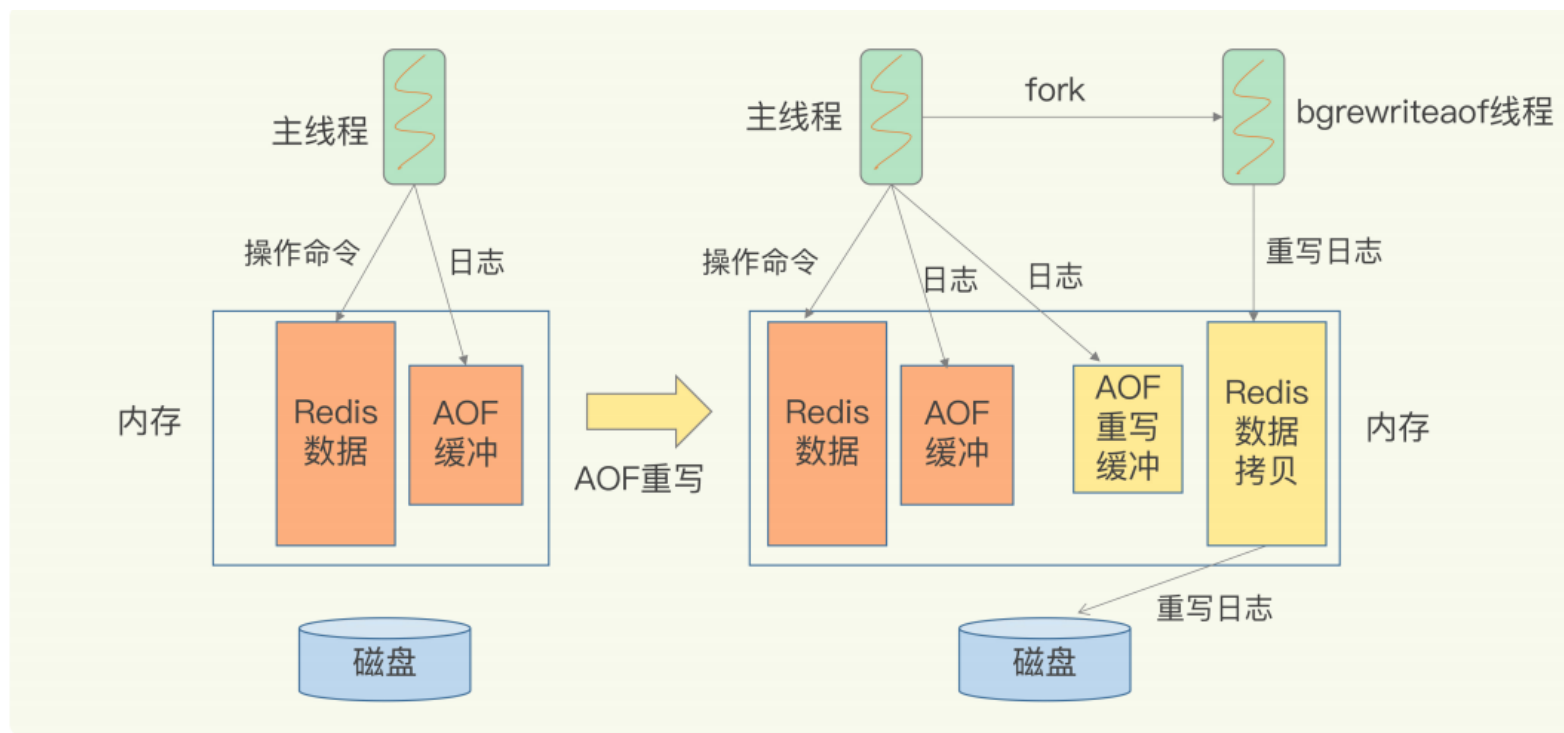
AOF 是以文件的形式在记录接收到的所有写命令。随着接收的写命令越来越多，AOF 文件会越来越大，所以注意AOF 文件过大带来的性能问题。

1. 文件系统本身对文件大小有限制，无法保存过大的文件；
2. 如果文件太大，之后再往里面追加命令记录的话，效率也会变低；

3. 如果发生宕机，AOF 中记录的命令要一个个被重新执行，用于故障恢复，如果日志文件太大，整个恢复过程就会非常缓慢，这就会影响到 Redis 的正常使用。

AOF 重写过程

1. 主线程 fork 出后台的bgrewriteaof 子进程，并把fork 会把主线程的内存（包含了最新数据）拷贝一份给 bgrewriteaof 子进程，子进程负责记入重写日志，fork子进程时，子进程是会拷贝父进程的页表，即虚实映射关系，而不会拷贝物理内存
2. 此时主线程并未阻塞，仍然可以接受请求，如果有写操作，Redis 会把这个操作写到AOF 日志的缓冲区，避免重写过程中宕机而造成的数据丢失。
3. 重写过程接受的写操作作也会被写到重写日志的缓冲区，等到拷贝数据的所有操作记录重写完成后，重写日志记录的这些最新操作也会写入新的 AOF 文件，以保证数据库最新状态的记录



AOF非阻塞的重写过程

AOF 重写的潜在风险

Redis采用fork子进程重写AOF文件时，潜在的阻塞风险包括：fork子进程 和 AOF重写过程中父进程产生写入bigkey的场景

1. fork子进程，fork这个瞬间一定是会阻塞主线程的。fork时并不会一次性拷贝所有内存数据给子进程，fork采用操作系统提供的写时复制(Copy On Write)机制，就是为了避免一次性拷贝大量内存数据给子进程造成的长时间阻塞问题，但fork子进程需要拷贝进程必要的的数据结构，其中有一项就是拷贝内存页表（虚拟内存和物理内存的映射索引表），这个拷贝过程会消耗大量CPU资源，拷贝完成之前整个进程是会阻塞的，阻塞时间取决于整个实例的内存大小，实例越大，内存页表越大，fork阻塞时间越久。拷贝内存页表完成后，子进程与父进程指向相同的内存地址空间，也就是说此时虽然产生了子进程，但是并没有申请与父进程相同的内存大小。那什么时候父子进程才会真正内存分离呢？“写时复制”顾名思义，就是在写发生时，才真正拷贝内存真正的数据，这个过程中，父进程也可能会产生阻塞的风险。

2. fork出的子进程指向与父进程相同的内存地址空间，此时子进程就可以执行AOF重写，把内存中的所有数据写入到AOF文件中。但是此时父进程依旧是会有流量写入的。如果父进程操作的是一个已经存在的key，那么这个时候父进程就会真正拷贝这个key对应的内存数据，申请新的内存空间，这样逐渐地，父子进程内存数据开始分离，父子进程逐渐拥有各自独立的内存空间。因为内存分配是以页为单位进行分配的，默认4k，如果父进程此时操作的是一个bigkey，重新申请大块内存耗时会变长，可能会产生阻塞风险。另外，如果操作系统开启了内存大页机制(Huge Page，页面大小2M)，那么父进程申请内存时阻塞的概率将会大大提高，所以在Redis机器上需要关闭Huge Page机制。Redis每次fork生成RDB或AOF重写完成后，都可以在Redis log中看到父进程重新申请了多大的内存空间。

混合持久化

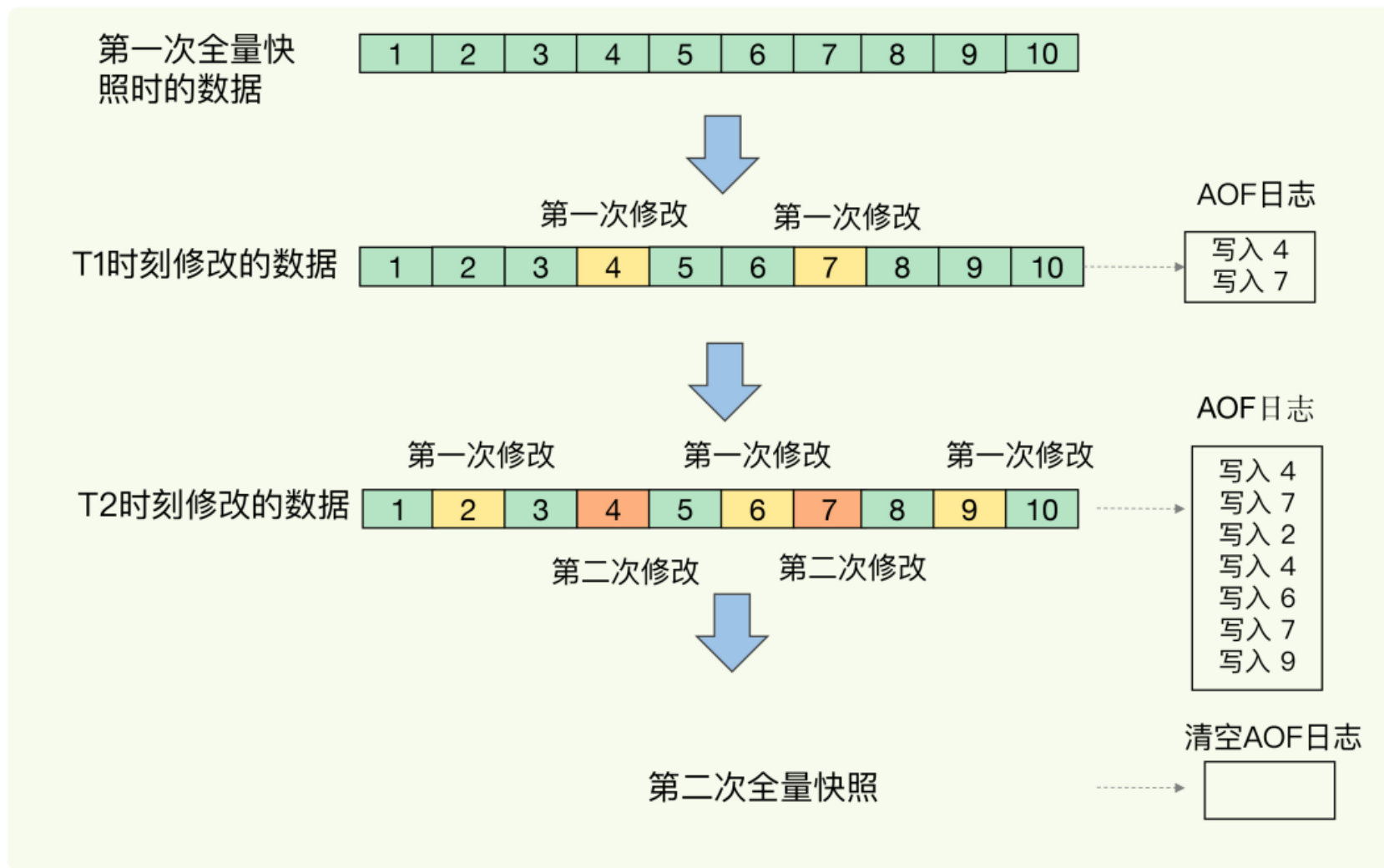
Redis 4.0 中提出了一个混合使用 AOF 日志和内存快照的方法。简单来说，内存快照以一定的频率执行，在两次快照之间，使用 AOF 日志记录这期间的所有命令操作。

混合持久化的好处

1. 快照不用很频繁地执行，这就避免了频繁 fork 对主线程的影响；
2. AOF日志也只用记录两次快照间的操作，就不会出现文件过大的情况了，也可以避免重写开销。

混合持久化的过程

1. T1 和 T2 时刻的修改，用 AOF 日志记录
2. 第二次全量快照时，可以清空 AOF 日志，此时数据都记录到快照中了，数据恢复时可以不依赖于日志了。



持久化的选择

1. 数据不能丢失时，内存快照和 AOF 的混合使用是一个很好的选择；
2. 如果允许分钟级别的数据丢失，可以只使用 RDB；

3. 如果只用 AOF，优先使用 everysec 的配置选项，因为它在可靠性和性能之间取了一个平衡